GCCFE prompting, task decomposition and reasoning-oriented prompting

1. What You'll Learn in This Section

In this lesson, you'll learn to:

- Write a single, well-rounded prompt that covers all five GCCFE elements without separate labels
- Add context and constraints that turn a vague request into a precise one
- Break a complex request into a step-by-step conversation with the AI
- Compare direct prompting with reasoning-oriented prompting and decompose a project into smaller tasks

2. Detailed Explanation

The GCCFE framework

GCCFE is a memory aid for the five elements of a good prompt. Some versions of this framework open with a role, but here the first element is the goal.

Letter	Element	What it means	Example
G	Goal	The exact task the AI should perform	"Create a quiz on Python"

C	Context	The background information the AI works with	"I'm a first year engineering student with no coding background."
C	Constraint	The restrictions or hard limits on the AI	"Please summarize this within 100 words."
F	Format	The shape of the output	Plain text or a table
E	Example	An optional sample of the desired output	A sample post in your own style

Examples are especially helpful when the output needs a personalised feel. Blogs, LinkedIn posts and emails are good cases.

Here is the part that trips people up. The five elements do not need separate labels inside your prompt. You can mix context, constraints and the rest anywhere in the text. A good prompt reads as one natural request that happens to cover every element.

Watch out for: writing a prompt as a list headed "Goal:", "Context:", "Constraint:" and so on. It works, but it is not the aim. Practice folding everything into a single request instead.

Why context matters

Context is the background you give the AI so it knows who it is helping and at what level. Without it, the AI has many random ways to respond.

Take the bare task "Create a quiz on Python". The AI might produce very beginner-friendly questions or very advanced ones, and any number of them. The prompt is missing three things:

- The target audience and level (beginner, intermediate or advanced)
- The number of questions
- The topics to be covered

Vague quiz prompts cause two more problems. The AI often gives repetitive questions at mixed difficulty. It also tends to reuse the same question while only renaming the data, for example "array a", then "array b", then "array c".

Now compare an improved version that adds context, a task, a format and a specific constraint:

```
I am a first year CS engineering student with no prior experience with programming.
The topics I have learned are if-else, loops and functions.
Create 10 moderate level questions on the above concepts to test my knowledge.
Please share a sample code with each one of them.
Make sure that each question is different from the other.
```

This is only one possible context, not a fixed template. The goal is to practice until writing prompts like this feels natural.

You can also ask for incremental complexity, which means each question is harder than the one before. Some exams work this way. Just add a line such as "Please increase the complexity incrementally, one question at a time."

Learner prompts in practice

A few sample prompts show how context plays out in different situations:

- A graduate with some Python knowledge listed loops, control statements and related topics. They asked for 10 moderate questions with simple code, distinct questions and consistent difficulty. It was well written but copied the model example too closely. A fresh context makes for better practice.
- A social media prompt asked for a post promoting a new AI-powered healthcare app. The app helps patients find doctors, book appointments and get basic health information. It targeted young adults in India, with simple language, no medical claims and under 100 words. The format was a catchy headline, the post and four hashtags, in a professional but friendly tone. The only fix was to drop the separate element labels and write one prompt.
- A 10th class student had learned about Indian independence, Mughal emperors and freedom fighters. They asked for 10 moderate questions, a scenario with each, and no mixing between them.
- A professional with two years of e-commerce experience who knows object-oriented programming asked for 20 in-depth questions, each with an answer and an explanation.

Why constraints matter

A constraint is a limit you place on the AI's output. Constraints narrow a generic request down to exactly what you need.

"Create a blog on the importance of upskilling" is too generic. Different platforms have different ways of writing content, so naming the platform helps. Upskilling also has no end, so the topic needs narrowing. Watch how the request sharpens as you add constraints one at a time:

1. Platform: write a LinkedIn post on the importance of upskilling.
2. Topic: narrow it to upskilling in AI. This is still a bit generic, but narrower.
3. Structure: at most seven bullet points.

4. Audience: only CS graduates with prior programming knowledge, a deliberately niche group.
5. Length: within 200 words.
6. Style: appropriate emojis wherever needed.

Constraints come in two flavours. Positive constraints tell the AI what it should do. Negative constraints tell it what not to do, in the form "do not do this".

Mixing elements freely: a system design plan

System design is an interview round in most software developer roles. A typical question asks how to design a WhatsApp chat engine, covering areas such as load testing. It spans both high-level and low-level design. One learner wrote this prompt to build a study plan:

```
Generate a structured plan to learn system design. I am an experienced software
developer and I want to improve my skills. Start with basic to advanced
concepts,
with detailed explanation, real-life use cases, examples, and animations or
visuals
where needed. Add an evaluation after each concept. Give me step-by-step topics
in
HTML format.
```

Notice there are no labels here. Context ("experienced software developer"), constraints ("basic to advanced", "evaluation after each concept") and format ("HTML format") sit side by side in plain sentences.

Breaking a complex task into smaller tasks

Very large requests, such as "give me a startup plan" or "give me 50 quiz questions", get worse as the output goes on. The first 10 to 20% of the answer is detailed. Toward the end the AI takes shortcuts. With 50 quiz questions, the first 10 to 15 get detailed explanations, but the last ones shrink to two- or five-line items.

The fix is to split the complex task into simpler ones. The technique is inspired by divide and conquer, which relates to the "divide and rule" policy the British applied in many countries.

A common beginner mistake is asking for 10 different things in one sentence. For example: "Analyze this business proposal and tell me everything about starting this as my new startup." This query is wide, complex and ambiguous, which simply means not clear.

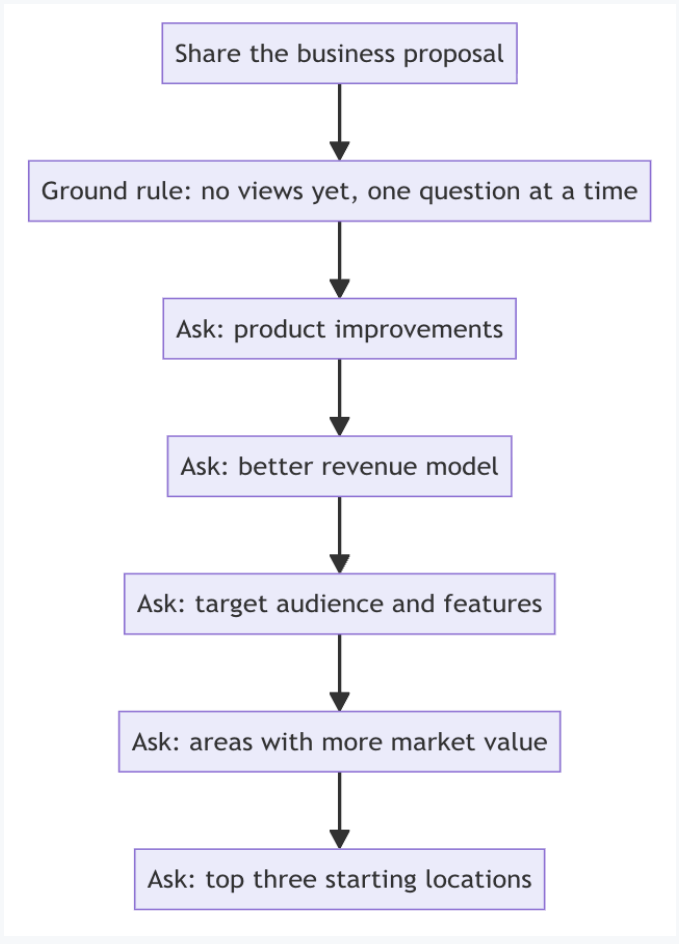
Worked example: analysing a business proposal

A better approach is to hold a conversation with the AI and decode one aspect at a time. The first message sets the ground rule:

```
Analyze the business proposal that I have shared. Don't share any views, as I
will
ask you one question at a time.
```

Then each follow-up tackles a single aspect:

- Please suggest improvements in the product described, at the product-level details.
- Any suggestions on the mentioned revenue models? Can you suggest a better one?
- Identify the most relevant target audience and features for the product.
- Suggest the area or location where the product has more market value, and the zones where it fits best.
- Give me the top three locations where the product will be best to start with.



Watch out for: skipping the ground rule when you share a document. Tell the AI explicitly not to give direct answers yet and to answer one question at a time. Otherwise it jumps straight to answers.

Practice task: analysing your resume

Every learner should ask an AI model to analyse their resume and suggest improvements, including an ATS score (how well the resume passes the automated test). The first step is to upload the resume document. Then break the analysis into separate requests.

One learner asked the AI to act as a "resume analyst" for a QA lead at a product organisation. The prompt asked for improvements to make the resume ATS friendly,

missing skills based on current market trends, and ATS-friendly keywords. The suggested improvements apply to almost any prompt:

- Skip "this is my resume" and your current role if the resume already shows that. Add it only when it is missing.
- Use a real, relevant job role for the persona. "Resume analyst" is not a common role. "Act as a senior tech recruiter with around 10 years of experience in hiring" gives better context.
- Remember that ATS friendliness is only one aspect. The resume that passes the automated test also goes to the hiring manager, so it must work for a human reader too.
- Ask which sections are missing and how the sections should be arranged.
- Add the job description to the prompt, because one resume does not work everywhere.

Two commonly quoted numbers explain why layout matters so much:

- About 74% of recruiters do not go to the second page of a resume.
- Recruiters spend on average around 6 to 7 seconds deciding whether to proceed with a resume.

Put together, the top few sections of your resume hold the most weight. A two-page resume is still fine for a very experienced person.

Tailoring pays off too. One candidate at IIT Bombay created around eight different resumes, two of them one page long. The placement portal allowed uploading several resumes and choosing one per application. Separate versions targeted roles such as software developer and data scientist. Some put achievements at the top and others put projects first, depending on the job description.

Reasoning-oriented prompting

Many real-world problems need several steps to reach an answer, not one direct decision:

Domain	Typical steps
Mathematics	Read the question, pick a formula, substitute values, calculate, get the final answer
Coding	Understand the problem, share the requirements, design an approach, implement, test, fix errors
Business decisions	Understand the problem, identify the data points, understand the constraints, compare options, conclude

This is where two prompting styles differ. Direct prompting gives the problem and asks only for the answer, such as "What is the solution to this problem?" Reasoning-oriented prompting gives the model the steps to follow. For a formula-based problem, you ask it to identify the variable values, calculate with them, and present the final answer in proper units. The model then solves the problem step by step.

Let's see both styles on the same question. First, the direct prompt:

```
A train travels 60 kilometers in one hour. How far will it travel in three hours?
Please give the final answer only.
```

The AI answers 180 kilometers, which is correct. Now the reasoning-oriented prompt:

```
A train travels 60 kilometers in one hour. How far will it travel in three hours?
Identify the speed from the distance and the time information.
Use the relationship between distance, speed and time to explain how the final answer will be computed using the formula.
Show the calculation as per this formula and present the answer with the relevant units.
```

This time the AI walks through its work:

1. It extracts the distance (60 km) and the time (1 hour).
2. It computes speed = distance / time = 60 / 1 = 60 km/h.
3. It calculates the distance for 3 hours as 60 × 3 = 180 km.

The final answer is the same. The difference is that every step is now visible, so you can check the logic rather than trust a bare number.

Task decomposition

Task decomposition means breaking a large problem into smaller problems. It takes the "one aspect at a time" idea and applies it to a whole project.

Consider the task "Build an AI application". A suggested decomposition looks like this:

1. Define the problem statement. Give the AI your random thoughts. It turns them into a proper problem statement and helps you choose a segment to focus on.
2. Identify the target audience who will use the application. Ask the AI for possible audiences.
3. Identify the sources of revenue. This step is very important.
4. Identify the potential risks involved.
5. Set the timeline and the steps to follow.

Try writing decompositions like this yourself rather than asking the AI to generate them. Doing it by hand builds your confidence.

Combined practice: planning a trip to Goa

This exercise brings GCCFE and task decomposition together on one request: "Plan a trip to Goa". Here is one strong learner prompt:

```
Plan a 3-day, 2-night trip to Goa. I'm a working professional with two friends.
We want to explore beaches, local attractions and try local food. Keep the
budget
below 15,000 per person, avoid activities requiring advanced booking, and use
affordable transportation and accommodation. Create a day-wise plan to visit
places
in South and North Goa. For each day provide activities in this format: morning
(place, activity, estimated cost), evening (food, club).
```

Let's break down what it does well and what it misses:

- "Plan a 3-day, 2-night trip" is both the task and a constraint.
- "Working professional with two friends" is context, not a constraint.
- The budget is covered, which is a very important factor in any travel plan.
- One constraint is missing: the diet type. Any request for food suggestions should state dietary restrictions such as veg or non-veg.

Another learner planned a family trip to Goa. The prompt asked for the best month for the trip and the best way to travel from the learner's location, such as by train or plane. That prompt should also say where each traveller lives. The planner may work in one city while family members live in another, and that context helps the AI.

Using AI as a mock interviewer

Many people use AI only for tips and tricks. A more powerful approach is to hold real discussions with it. One strong use case is asking the AI to act as a mock interviewer, a practice interviewer that questions you like a real one would.

A mock interview should mimic a realistic situation. It runs in conversation format, where the previous output decides the next input. A real interviewer carries forward from your last answer and digs deeper into it. Your mock interview should connect in the same way.

Watch out for: giving all your details at once and just asking for interview tips. That gives you a list, not an interview.

Making practice a habit

Practice means applying the GCCFE framework and task decomposition every time you use AI, not repeating the same questions. As a small side tip, the keyboard shortcut to turn lines into comments is Ctrl + / (Cmd + / on Mac).

3. Key Takeaways

- GCCFE (Goal, Context, Constraint, Format, Example) covers the five elements of a good prompt, written as one natural request without separate labels.
- Context and constraints, both positive and negative, turn a vague request into precise, non-repetitive output.
- Large requests degrade toward the end, so split them and work through the AI one aspect at a time after setting a ground rule.
- Reasoning-oriented prompting gives the model the steps to follow, so its work becomes visible instead of a bare answer.
- Task decomposition applies the same splitting to whole projects, and conversation-style prompting powers are used like mock interviews.

Mental model: Think of prompting as divide and conquer: cover every GCCFE element in each request, then split big problems into small steps the AI can handle one at a time.